The Effect of Sampling Temperature on Problem-Solving in Large Language Models

Matthew Renze
Johns Hopkins University
mrenze1@jhu.edu

Abstract

In this research study, we empirically investigate the effect of sampling temperature on the performance of Large Language Models (LLMs) on various problem-solving tasks.

We created a multiple-choice question-and-answer (MCQA) exam by randomly sampling problems from standard LLM benchmarks. Then, we used four popular LLMs with five prompt-engineering techniques to solve the MCQA problems while increasing the sampling temperature from 0.0 to 1.0.

Despite anecdotal reports to the contrary, our empirical results indicate that temperature *does not* have a statistically significant impact on LLM performance for problem-solving tasks. In addition, these results appear to hold regardless of the LLM, the prompt-engineering technique, or the problem domain.

These results have practical implications for AI systems engineers using LLMs to create automated AI systems. In addition, our results provide more general insight for AI researchers on the role of temperature-based sampling in model hallucination and solution-space search.

All code, data, and supplemental materials are available on GitHub[1].

Keywords

Large Language Model (LLM), Generative Pretrained Transformer (GPT), sampling temperature

1. Introduction

1.1 Background

In recent years, Large Language Models (LLMs) have revolutionized the field of artificial intelligence. Open-source LLMs and pay-per-use APIs have allowed engineers to incorporate LLMs in their AI systems. However, prompt engineering and hyperparameter tuning are required to work effectively with LLMs.

Prompt-engineering techniques help LLMs solve complex problems, avoid hallucinations, and provide more accurate responses. For example, we can use techniques like chain-of-thought, tree-of-thought, self-criticism, and self-consistency to improve LLM performance [1], [2].

[1] https://github.com/matthewrenze/jhu-llm-temperature/

In addition, several inference hyperparameters can be adjusted to change the LLM's output at runtime. For example, hyperparameters like sample temperature, top-k sampling, repetition penalty, and maximum token length all affect the LLMs output and performance [3]–[6].

Despite significant interest in LLMs and progress in LLM best practices, many open questions remain about optimal prompt-engineering techniques and inference hyperparameters for LLMs. To complicate matters, various local optima may exist for each problem type and problem domain [6].

The prompt-engineering community has an abundance of opinions and anecdotal evidence regarding optimal prompt-engineering techniques and inference hyperparameter settings. However, we currently lack systematic studies and empirical evidence to support many of these claims.

As a result, this paper aims to address the open question of the optimal LLM sampling temperature for problem-solving tasks. In addition, we aim to provide a systematic study with empirical results to add to the growing body of knowledge used to create LLM and prompt-engineering best practices.

1.2 – Sampling Temperature

Sampling temperature is a hyperparameter of an LLM used in a temperature-based sampling process. It controls the randomness of the model's output at inference time [6]–[9].

At each step of an LLM's decoding process, the LLM uses the previous tokens to choose the next output token. The final layer of the LLM uses a softmax function to convert raw scores (logits) into probabilities.

In greedy sampling, the model will always choose the most likely next token. However, for probabilistic sampling, the next token is selected from a probability distribution.

Temperature sampling is a modification to the softmax function, which adjusts the resulting probability mass functions. In this modified softmax function, v_k is the k-th vocabulary token, l_k is the token's logit, and τ is a constant temperature.

$$\Pr(v_k) = \frac{e^{l_k/\tau}}{\sum_i e^{l_i/\tau}}$$

Equation 1 - Equation for temperature-based sampling

A lower temperature makes the output of the LLM more deterministic, thus favoring the most likely predictions. This conservativeness is captured by the model's tendency to produce more repetitive, focused, and less diverse output based on the patterns most commonly seen in the training data [6]–[8].

A higher temperature makes the output more random, thus favoring more "creative" predictions. This creativity is captured by the models' willingness to explore more unconventional and less likely outputs. Higher temperatures can lead to novel text, diverse ideas, and creative solutions to problems [6]–[8].

In the context of problem-solving, temperature can be seen as a trade-off between exploring and exploiting possible solutions within the solution space. Higher temperatures explore a solution space more broadly; lower temperatures tend to exploit known solutions in a problem's solution space.

1.3 Choosing a Sampling Temperature

Within the prompt-engineering community, there are a variety of opinions and best practices regarding the ideal sampling temperature for various problem-solving tasks [10], [11].

Low sampling temperatures are recommended for tasks requiring precision and factual accuracy, such as technical writing, code generation, or question-answering [12], [13]. However, higher temperatures are recommended for tasks requiring creativity, such as writing poetry, creating stories, or brainstorming.

Higher temperatures also increase the probability of model hallucination. Hallucination is a phenomenon where an LLM produces statistically probable responses that are factually incorrect or nonsensical. As a result, optimal temperature selection is also a balance between creativity and hallucination [14].

Practical guidelines for choosing a sampling temperature for a specific task or problem domain are often vague or anecdotal. Prompt engineering guides often provide hypothetical examples of optimal sample temperatures for various tasks. However, they rarely cite any sources or provide empirical evidence.[2]

As a result, the current state of choosing the optimal sample temperature for specific problems is largely based on guesswork, gut instinct, non-systematic experimentation, and iterative refinement.[3]

1.4 Hypothesis

Given the lack of strong evidence for best practices regarding optimal sampling temperature, our research study empirically investigated the effect of LLM sampling temperature, specifically on problem-solving tasks. To test these claims, we formulated the following hypotheses:

Null Hypothesis (H_0): Increasing or decreasing sampling temperature has no effect on the problem-solving performance of the LLM.

Alternative Hypothesis (H_1): Increasing or decreasing sampling temperature improves the problem-solving performance of the LLM agent for specific types of problems.

For both hypotheses, problem-solving performance was measured by the LLM's accuracy in correctly answering 1,000 multiple-choice questions across ten problem domains, ten times each. To determine the validity of these hypotheses, we conducted a hypothesis test using a significance level (α) of 0.05.

2. Methods

2.1 Data

The test dataset used in this research study consists of a series of Multiple-Choice Question-and-Answer (MCQA) exams derived from commonly used LLM performance benchmarks.

First, we reviewed the prior literature to identify benchmarks frequently used to evaluate LLMs. We limited our candidate benchmarks to those containing MCQA problems so that we could use correct-answer accuracy as our primary performance metric.

Next, we selected a set of problems that covered a range of problem-solving domains (e.g., math, science, law, etc.) In addition, we selected problem sets that spanned a wide range of difficulty levels

---

[2] A few empirical studies exist that indicate sampling temperature does have an affect on LLM performance on some types of problem-solving tasks (e.g., code generation, engineering exams, etc.) [12], [13], [35]

[3] Even the GPT-4 Technical Report explains that the authors used their "best-guess" when choosing sampling temperatures while evaluating GPT-4 on various benchmarks. See Appendix A in the GPT-4 Technical Report. [4].

(e.g., primary school, secondary school, university). These problem sets can be seen in Table 1.

MCQA Exam Data Sources				
Problem Set	**Benchmark**	**Domain**	**Questions**	**Source**
ARC Challenge Test	ARC	Science	1,173	[15]
AQUA-RAT	AGI Eval	Math	254	[16]
Hellaswag Val	Hellaswag	Common Sense Reasoning	10,042	[17]
LogiQA (English)	AGI Eval	Logic	651	[16], [18]
LSAT-AR	AGI Eval	Law (Analytic Reasoning)	230	[16], [19]
LSAT-LR	AGI Eval	Law (Logical Reasoning)	510	[16], [19]
LSAT-RC	AGI Eval	Law (Reading Comprehension)	260	[16], [19]
MedMCQA Valid	MedMCQA	Medicine	6,150	[20]
SAT-English	AGI Eval	English	206	[16]
SAT-Math	AGI Eval	Math	220	[16]

Table 1 - Source of problem sets used to create the multi-domain MCQA exam.

Then, we converted the benchmark problems from their original data format into a standardized data structure using the JSON Lines (JSON-L) format [21]. Our standardized set of exams allowed us to use the exams interchangeably without modifying the code in the test harness. See Figure 1 for a sample.

```
{
  "source": "arc/arc-challenge-test",
  "source_id": 1,
  "topic": "Science",
  "context": "",
  "question": "An astronomer observes that a planet rotates faster after a meteorite impact.
               Which is the most likely effect of this increase in rotation?",
  "choices": {
    "A": "Planetary density will decrease.",
    "B": "Planetary years will become longer.",
    "C": "Planetary days will become shorter.",
    "D": "Planetary gravity will become stronger." },
  "answer": "C",
  "solution":""
}
```

Figure 1 - Sample of a multiple-choice question in JSON-L format – with whitespace added for readability.

Finally, we created two MCQA exams of different sizes. We created a small exam of 100 questions (total) by randomly sampling ten questions from each of the ten domain-specific problem sets. This 100-question (small) exam was used for our high-level analysis of sampling temperature across all models.

However, we also created a larger exam with 1,000 questions by randomly sampling 100 problems from each of the ten problem sets. This 1,000-question (large) exam was used exclusively with GPT-3.5 for our more detailed analysis of temperature across problem domains and prompt types.

2.2 Models

The LLM models used in this research project consist of four commonly used foundational LLMs. To complement our analysis, we also performed experiments using five prompts created with commonly used prompt-engineering techniques.

First, we reviewed the prior literature to identify candidate LLMs commonly used for problem-solving tasks. We limited our candidate models to those that allowed the model temperature to be specified via their API. These LLMs included GPT-3.5, GPT-4, Llama 2 7B, and Llama 2 70B [4], [5], [22], [23].

Next, we reviewed the existing literature for commonly used prompt-engineering techniques. We limited our candidate prompts to those that could be performed in a single request-and-response cycle with one-shot in-context learning. We excluded multi-step agents, few-shot learning, and model fine-tuning.

As a result, we selected the following five prompt-engineering techniques to construct our system prompts:

- **Baseline** - no prompt engineering; the LLM is instructed to return only a single multiple-choice answer as its output (e.g., 'Answer("C")').
- **Domain Expertise** – the system prompt specifies that the LLM is an expert in the problem domain of the exam (e.g., "medicine") or the topic of the problem (e.g., "anatomy") [2].
- **Self-recitation** – the system prompt instructs the LLM to recite its own internal knowledge about the problem before answering the question [2], [24].
- **Chain-of-Thought (CoT)** – the system prompt instructs the LLM to "think step-by-step" to encourage it to reason through the problem procedurally [25], [26].
- **Self-criticism** – the system prompt instructs the LLM to critically evaluate its previous thoughts to identify any errors in facts, logic, and reasoning [27], [28].[4]
- **Composite** – the system prompt combines domain expertise, self-recitation, chain-of-thought, and self-criticism.

Finally, we provided the LLM with a single example problem-and-solution pair for one-shot in-context learning. The example solution was adapted for each prompt based on the prompt-engineering technique used. For example, the self-recitation prompt included a knowledge recitation in its solution.

[4] The self-criticism prompt cannot be independently tested because self-criticism requires a knowledge recitation or a chain of thought to be critically evaluated.

```
[System Prompt]
You are an expert in {{expertise}}.
Your task is to answer the following multiple-choice questions.
First, you should recite all of the relevant knowledge you have about the question and each
option.
Next, you should think step-by-step through the problem to ensure you have the correct answer.
Then, you should critically evaluate your thoughts to identify any flaws in your facts, logic,
and reasoning.
Finally, you MUST answer the question using the following format 'Action: Answer("[choice]")'
The parameter [choice] is the letter or number of the answer you want to select (e.g. "A",
"B", "C", or "D")
For example, 'Answer("C")' will select choice "C" as the best answer.
The answer MUST ALWAYS be one of the available choices; it CANNOT be "None of the Above".
If you think the answer is "none of the above", then you MUST select the most likely answer.

[Example Problem]
Question: What is the capital of the state where Johns Hopkins University is located?
Choices:
  A: Baltimore
  B: Annapolis
  C: Des Moines
  D: Las Vegas

[Example Solution]
Knowledge:
  Johns Hopkins University is located in Baltimore, Maryland.
  A: Baltimore is a city located in the state of Maryland, but it is not the capital of
Maryland.
  B: Annapolis is a the capital of the State of Maryland.
  C: Des Moines is a city located in the State of Iowa, but it is not the capital of Iowa.
  D: Las Vegas is located in the State of Nevada, but it is not the capital of Nevada.
Thought:
  Johns Hopkins University is located in Baltimore.
  Baltimore is a city located in the state of Maryland.
  The capital of Maryland is Baltimore.
  Therefore, the capital of the state where Johns Hopkins University is located is Baltimore.
  The answer is A: Baltimore.
Criticism:
  You are correct that Johns Hopkins is located in the State of Baltimore.
  However, the capital of Maryland is Annapolis, not Baltimore.
  So, the correct answer is actually B: Annapolis.
Action: Answer("B")
```

Figure 2 - Sample of system prompt and one-shot example (i.e., problem-and-solution pair)

2.3 Metrics

To test our hypothesis, we measured the LLM's correct-answer accuracy as our primary performance metric. To further support our findings, we also measured the similarity of the LLM's responses using a series of text-similarity metrics.

These metrics are defined as follows:

- **Accuracy** – the ratio of MCQA problems that the LLM correctly solved
- **Jaccard similarity** – the ratio of the intersection to the union of word sets in the output text [29]
- **Bag-of-Words (BoW) similarity** – comparison of the frequency of each word, ignoring order [30]
- **TF-IDF similarity** – comparison of word frequency weighted by inverse document frequency [31]

- **Levenshtein similarity** – the number of edits needed to change one text into the other [32]
- **BLEU score** – comparison of similarity based on n-gram overlap [33]
- **SBERT similarity** – semantic similarity computed using Sentence-BERT embeddings [34]

2.4 Analysis

We performed a repeated-measures ANOVA test to assess the statistical significance of these results at a significance level of 0.05. We performed the analysis across temperatures ranging from 0.0 to 1.0. in increments of 0.1 using GPT-3.5 and GPT-4, the composite prompt, and the 100-question exam.

3. Results

3.1 Accuracy vs. Temperature

Our analysis revealed that the mean accuracy remained relatively stable across all sampling temperatures.

The repeated-measures ANOVA for GPT-3.5 yielded $F(10, 990) = 0.2858$, $p = 0.9844$. The GPT-4 ANOVA yielded $F(10, 990) = 0.8763$, $p = 0.5550$. The two Llama models did not perform significantly better than statistically random chance, so their results had to be excluded from our analysis.

Given a significance level of 0.05, the high p-values indicate that the differences in accuracy across temperatures were not statistically significant. Thus, we found no evidence to reject the null hypothesis. These results suggest that sampling temperature does not affect LLM performance in problem-solving.

Visual examination of the data also supported these findings. The line charts in Figure 3 show no discernable trend in problem-solving accuracy as a function of temperature. These results appear to hold regardless of the specific model, system prompt, and problem domain, as seen in Figure 4.

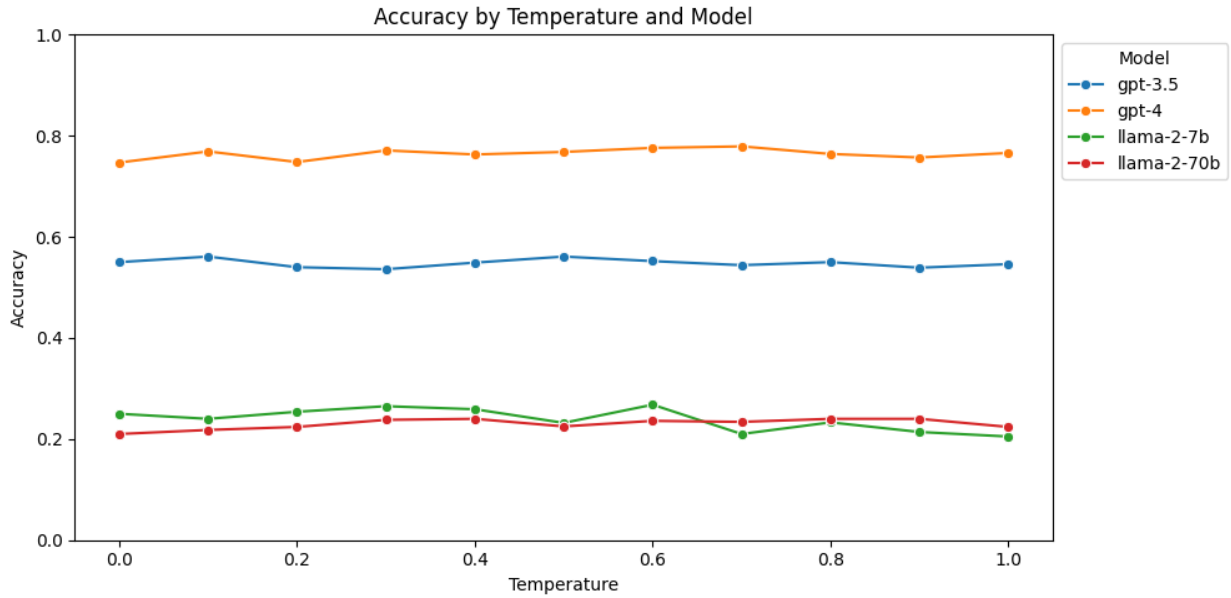

Figure 3 - Problem-solving accuracy remains stable with sampling temperature for all four LLMs on the 100-question comprehensive exam.

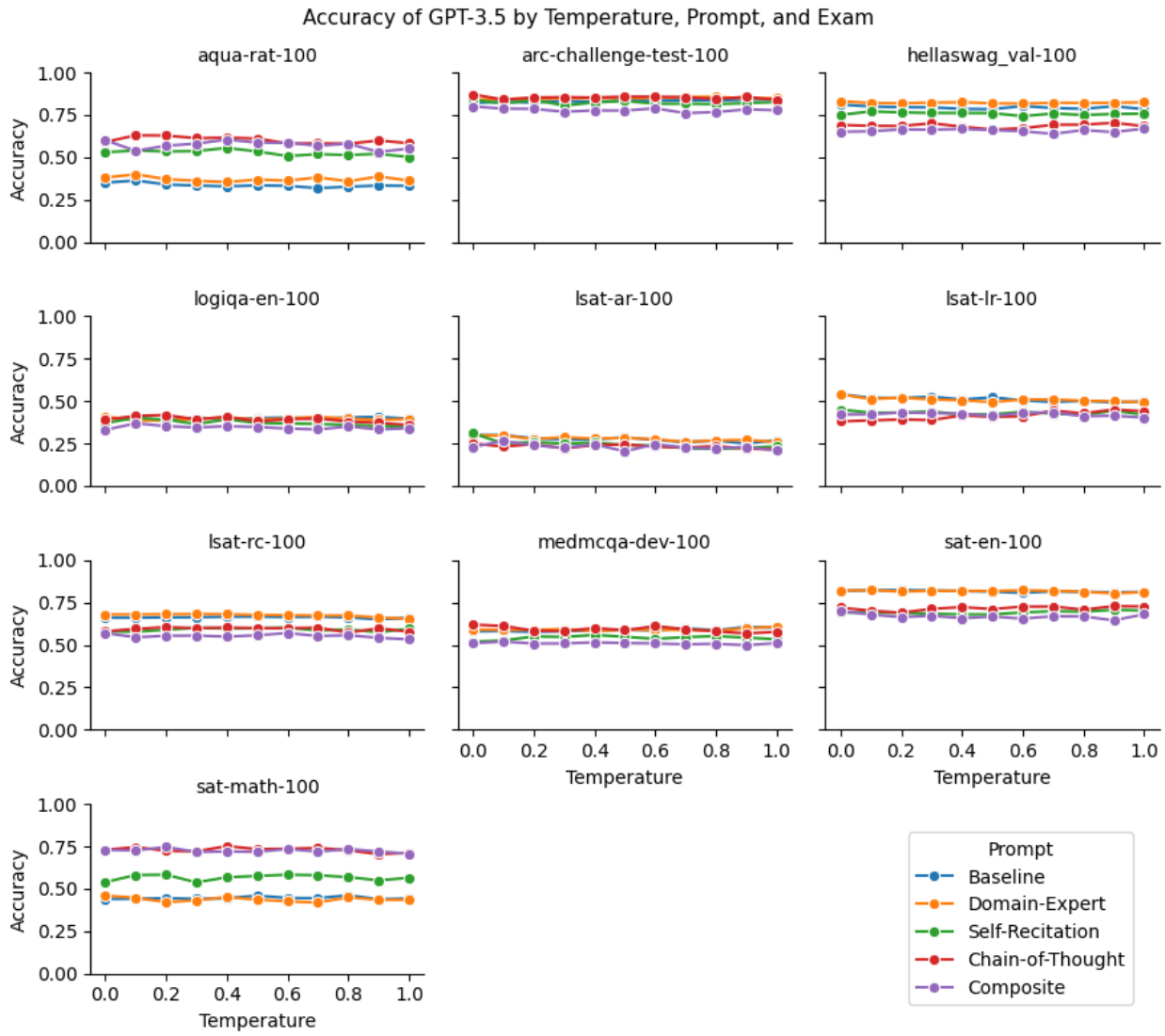

Figure 4 - The accuracy of GPT-3.5 remains relatively stable across all temperatures regardless of the system prompt or the problem domain on all 1,000 multiple-choice questions.

3.2 Text Variability vs. Temperature

To further support our results, we analyzed text variability as a function of temperature. Our results show a clear trend of decreasing text similarity (thus increasing text variability) as temperature increases. This confirms that the sampling-temperature hyperparameter was behaving as expected.

These results are consistent with our understanding of sampling temperature, indicating that higher temperatures produce more widely varied outputs – as seen in Figure 5. In addition, these results hold regardless of the LLM model, prompt-engineering technique, or problem domain – as seen in Figure 6.

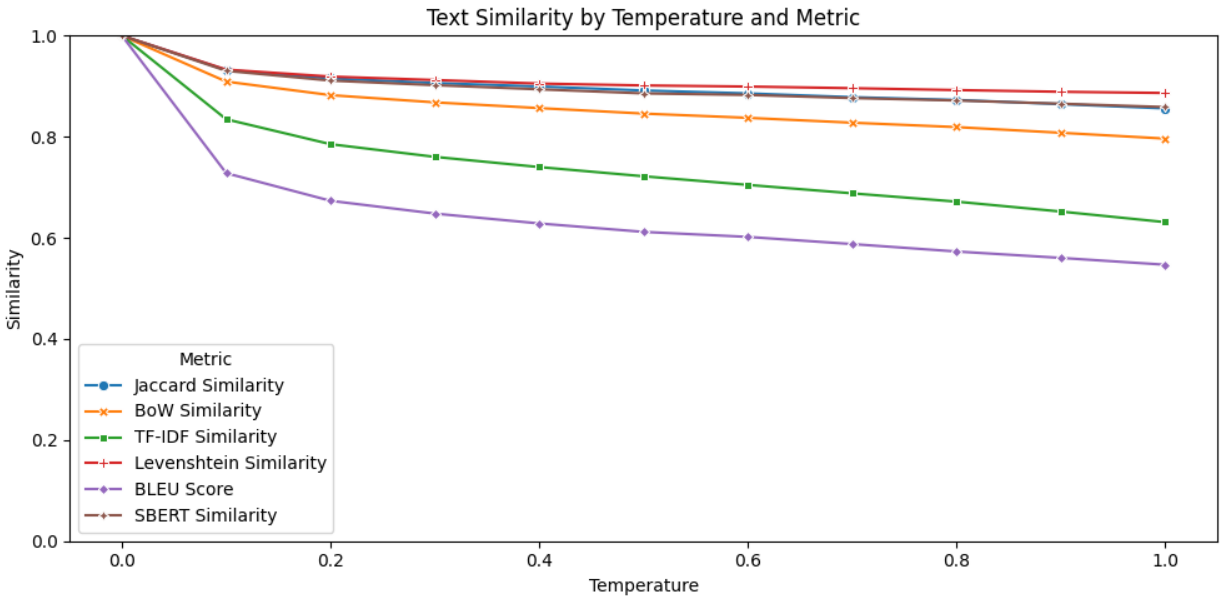

Figure 5 - Text similarity decreases as temperature increases for GPT-3.5 on all 1,000 questions. This effect can be seen using all six text-similarity metrics. These results confirm that temperature-based sampling behaved correctly during our experiments.

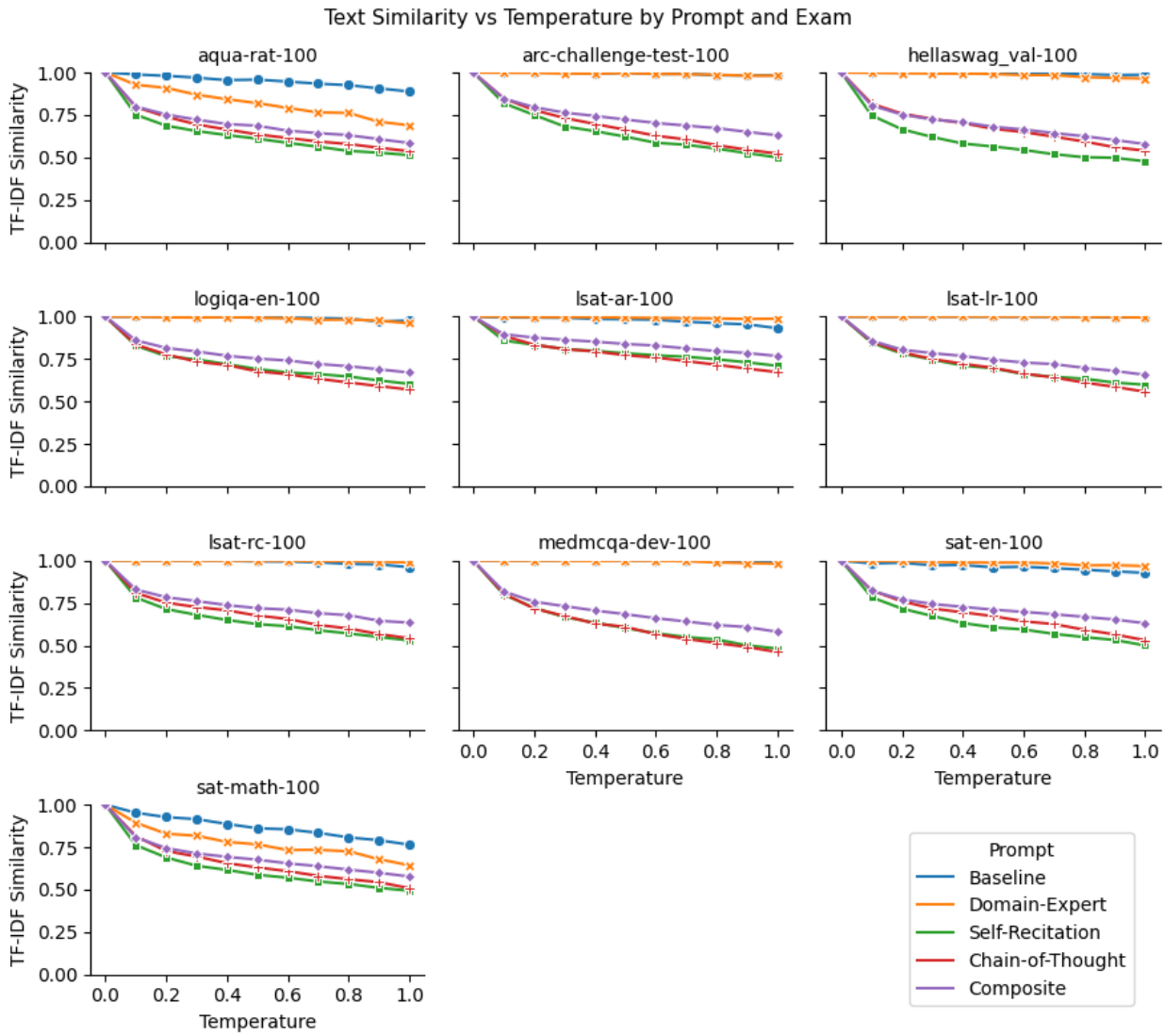

Figure 6 - TF-IDF text similarity decreases as temperature increases regardless of the prompt or problem domain. The baseline and domain-expert prompts produce short LLM responses, so their decrease in text similarity across temperatures is minimal.

4. Discussion

4.1 Limitations

There were several limitations to our research study:

First, our study was limited to a small subset of problem-solving tasks and problem domains. As a result, the effects of sampling temperature on other types of problems may have eluded our analysis.

Second, we only explored a small subset of prompt-engineering techniques using a single prompt-and-response cycle with one-shot in-context learning. As a result, there may be more complex prompts or agent architectures that better leverage sampling temperature in their problem-solving capabilities.

Third, due to cost considerations, we could only fully explore the landscape of system prompts and problem domains with GPT-3.5. As a result, there may exist other combinations of system prompts and problem domains where other LLMs may better utilize sampling temperature for problem-solving.

Finally, our research study explored a range of temperatures from 0.0 to 1.0. However, some LLMs allow the temperature hyperparameter to be set higher than 1.0. As a result, temperatures higher than 1.0 may produce statistically significant changes in accuracy not seen in our analysis.

4.2 Implications

This research study provided empirical evidence that sampling temperature does not affect the problem-solving capabilities of LLMs. These results contradict the current belief held by some that optimal temperatures exist for solving specific types of problems across various problem domains.

Addressing this question may save AI systems engineers from expending significant time and resources testing agents using various sampling temperature values. In addition, it may reduce needless debate in the prompt-engineering community about optimal temperatures for specific problem-solving tasks.

This research also provides more general insight for AI researchers studying the effects of sampling temperature on LLM hallucination and solution-space search. In the case of MCQA problems, increasing temperature up to 1.0 does not appear to increase LLM hallucinations or improve solution-space search.

4.3 Future Research

To improve upon this research, we propose the following follow-up experiments:

First, we should perform additional experiments with other commonly used LLM benchmarks, problem-solving tasks, and problem domains. We recommend specifically targeting tasks and problem domains that require more creative solutions or lateral “out-of-the-box” thinking.

Second, we should extend the range of the sampling temperature beyond 1.0 to see what effects higher temperatures might have. However, based on a quick experiment we ran with GPT-3.5, performance drops off sharply after a temperature of 1.2 until it reaches random-chance performance at 1.4.

Third, we recommend expanding beyond MCQA problems to other types of problems whose correct answers are more open-ended. The limited effects of sampling temperature in our experiments may have simply resulted from the constraints imposed by the structure of MCQA problems.

Finally, we recommend a more in-depth error analysis to determine if any sub-types of problems within problem domains benefit from changes to sampling temperature. It is possible that statistical noise or averaging may have hidden individual problems that were sensitive to changes in sampling temperature.

5. Conclusion

This research study provided an empirical investigation of the effect of sampling temperature on the problem-solving performance of LLMs across multiple problem domains.

We demonstrated that changes in sampling temperature within the range of 0.0 to 1.0 do not produce statistically significant differences in problem-solving performance across multiple LLMs, prompt-engineering techniques, and problem domains.

Our findings challenge the prevailing belief that temperature plays an essential role in optimizing an LLM's performance for problem-solving tasks within specific problem domains.

These results have practical implications for AI systems engineers using LLMs to create new AI systems. In addition, they have theoretical implications for AI researchers studying model hallucination and solution-space search with LLMs.

6. References

[1] G. Mialon *et al.*, "Augmented Language Models: a Survey," *ArXiv*, Feb. 2023, Accessed: Apr. 28, 2023. [Online]. Available: https://arxiv.org/abs/2302.07842v1

[2] J. White *et al.*, "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT," *ArXiv*, Feb. 2023, Accessed: Nov. 26, 2023. [Online]. Available: https://arxiv.org/abs/2302.11382

[3] Open AI, "https://platform.openai.com/docs/api-reference/chat/create," Open AI - API Reference. Accessed: Nov. 25, 2023. [Online]. Available: https://platform.openai.com/docs/api-reference/chat/create

[4] Open AI, "GPT-4 Technical Report," *ArXiv*, Mar. 2023, Accessed: Apr. 28, 2023. [Online]. Available: https://arxiv.org/abs/2303.08774

[5] H. Touvron *et al.*, "Llama 2: Open Foundation and Fine-Tuned Chat Models," *ArXiv*, Jul. 2023, Accessed: Dec. 07, 2023. [Online]. Available: https://arxiv.org/abs/2307.09288

[6] C. Wang, S. X. Liu, and A. H. Awadallah, "Cost-Effective Hyperparameter Optimization for Large Language Model Generation Inference," *arXiv*. 2023. Accessed: Nov. 22, 2023. [Online]. Available: https://arxiv.org/abs/2303.04673

[7] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," *ArXiv*, Mar. 2015, Accessed: Dec. 07, 2023. [Online]. Available: https://arxiv.org/abs/1503.02531

[8] P.-H. Wang *et al.*, "Contextual Temperature for Language Modeling," *ArXiv*, Dec. 2020, Accessed: Nov. 26, 2023. [Online]. Available: https://arxiv.org/abs/2012.13575

[9] D. H. Ackley, G. E. Hinton, and T. J. Sejnowski, "A Learning Algorithm for Boltzmann Machines," *Cogn Sci*, vol. 9, no. 1, pp. 147–169, 1985, doi: https://doi.org/10.1016/S0364-0213(85)80012-4.

[10] J. Shieh, "Best Practices for Prompt Engineering with OpenAI API," OpenAI. Accessed: Nov. 26, 2023. [Online]. Available: https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-openai-api

[11] Microsoft, "Completions - Learn How to Generate or Manipulate Text," Azure OpenAI Service Documentation. Accessed: Nov. 26, 2023. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/openai/how-to/completions

[12] Y. Zhu *et al.*, "Improving Code Generation by Dynamic Temperature Sampling," *ArXiv*, Sep. 2023, Accessed: Nov. 26, 2023. [Online]. Available: https://arxiv.org/abs/2309.02772

[13] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A Systematic Evaluation of Large Language Models of Code," in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine*

Programming, in MAPS 2022. New York, NY, USA: Association for Computing Machinery, 2022, pp. 1–10. doi: 10.1145/3520312.3534862.

[14] M. Lee, "A Mathematical Investigation of Hallucination and Creativity in GPT Models," *Mathematics*, vol. 11, no. 10, p. 2320, May 2023, doi: 10.3390/math11102320.

[15] P. Clark *et al.*, "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," *ArXiv*, Mar. 2018, Accessed: Nov. 22, 2023. [Online]. Available: https://arxiv.org/abs/1803.05457

[16] W. Zhong *et al.*, "AGIEval: A Human-Centric Benchmark for Evaluating Foundation Models," *ArXiv*, Apr. 2023, Accessed: Dec. 07, 2023. [Online]. Available: https://arxiv.org/abs/2304.06364

[17] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a Machine Really Finish Your Sentence?," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019. Accessed: Nov. 22, 2023. [Online]. Available: https://arxiv.org/abs/1905.07830

[18] J. Liu, L. Cui, H. Liu, D. Huang, Y. Wang, and Y. Zhang, "LogiQA: A Challenge Dataset for Machine Reading Comprehension with Logical Reasoning," in *International Joint Conference on Artificial Intelligence*, 2020. Accessed: Nov. 22, 2023. [Online]. Available: https://arxiv.org/abs/2007.08124

[19] S. Wang *et al.*, "From LSAT: The Progress and Challenges of Complex Reasoning," *IEEE/ACM Trans Audio Speech Lang Process*, vol. 30, pp. 2201–2216, Aug. 2021, Accessed: Nov. 22, 2023. [Online]. Available: https://doi.org/10.1109/TASLP.2022.3164218

[20] A. Pal, L. K. Umapathi, and M. Sankarasubbu, "MedMCQA: A Large-scale Multi-Subject Multi-Choice Dataset for Medical domain Question Answering," in *Proceedings of the Conference on Health, Inference, and Learning*, PMLR, 2022, pp. 248–260. Accessed: Nov. 22, 2023. [Online]. Available: https://proceedings.mlr.press/v174/pal22a.html

[21] I. Ward, "JSON Lines." Accessed: Nov. 22, 2023. [Online]. Available: https://jsonlines.org/

[22] Open AI, "Introducing ChatGPT," Open AI. Accessed: Apr. 28, 2023. [Online]. Available: https://openai.com/blog/chatgpt

[23] Open AI, "GPT-4," Open AI. Accessed: Apr. 28, 2023. [Online]. Available: https://openai.com/research/gpt-4

[24] Z. Sun, X. Wang, Y. Tay, Y. Yang, and D. Zhou, "Recitation-Augmented Language Models," in *The Eleventh International Conference on Learning Representations*, Oct. 2023. Accessed: Nov. 22, 2023. [Online]. Available: https://openreview.net/forum?id=-cqvvvb-NkI

[25] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large Language Models are Zero-Shot Reasoners," in *Advances in Neural Information Processing Systems*, May 2022, pp. 22199–22213. Accessed: Dec. 07, 2023. [Online]. Available: https://arxiv.org/abs/2205.11916

[26] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *ArXiv*, Jan. 2022, Accessed: Dec. 07, 2023. [Online]. Available: https://arxiv.org/abs/2201.11903

[27] R. Wang, H. Wang, F. Mi, Y. Chen, R. Xu, and K.-F. Wong, “Self-Critique Prompting with Large Language Models for Inductive Instructions,” *ArXiv*, May 2023, Accessed: Nov. 22, 2023. [Online]. Available: https://arxiv.org/abs/2305.13733

[28] S. Huo, N. Arabzadeh, and C. L. A. Clarke, “Retrieving Supporting Evidence for Generative Question Answering,” *ArXiv*, Sep. 2023, doi: 10.1145/3624918.3625336.

[29] P. Jaccard, “The Distribution of Flora in the Alpine Zone,” *New Phytologist*, vol. 11, no. 2, pp. 37–50, Feb. 1912, doi: 10.1111/j.1469-8137.1912.tb05611.x.

[30] Z. S. Harris, “Distributional Structure,” *WORD*, vol. 10, no. 2–3, pp. 146–162, Aug. 1954, doi: 10.1080/00437956.1954.11659520.

[31] K. Spärck Jones, “A Statistical Interpretation of Term Specificity and Its Application in Retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, Jan. 1972, doi: 10.1108/eb026526.

[32] V. I. Levenshtein, “Binary Codes Capable of Correcting Deletions, Insertions and Reversals,” *Soviet Physics Doklady*, vol. 10, pp. 707–710, 1966, Accessed: Nov. 22, 2023. [Online]. Available: https://nymity.ch/sybilhunting/pdf/Levenshtein1966a.pdf

[33] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU,” in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics - ACL ’02*, Morristown, NJ, USA: Association for Computational Linguistics, 2001, p. 311. doi: 10.3115/1073083.1073135.

[34] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, Aug. 2019. Accessed: Nov. 22, 2023. [Online]. Available: https://arxiv.org/abs/1908.10084

[35] V. Pursnani, Y. Sermet, and I. Demir, “Performance of ChatGPT on the US Fundamentals of Engineering Exam: Comprehensive Assessment of Proficiency and Potential Implications for Professional Environmental Engineering Practice,” *ArXiv*, Apr. 2023, Accessed: Dec. 07, 2023. [Online]. Available: https://arxiv.org/abs/2304.12198